

Reviewer Pack — TP_2 Minor-Sign Defect of Spectral Pseudo-Moment Matrix Boun...

Entry 16809e692923 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-17 09:29:28 UTC

TP_2 Minor-Sign Defect of Spectral Pseudo-Moment Matrix Bounds Max-CUT Ratio

Entry ID: 16809e692923 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-17 09:29:28 UTC

1. Conjecture

Field A (mathematical branch): Karlin-Edrei-Schoenberg total positivity / Pólya frequency theory (TP_k matrices, sign-regularity of 2x2 minors) **Field B** (complexity object): Degree-2 SOS pseudo-moment matrix for Max-CUT on graphs (Goemans-Williamson regime), with a lifting interpretation via gadget composition

Statement:

For a connected graph $G=(V,E)$ on n vertices with combinatorial Laplacian $L=D-A$, let $v_2, v_3 \in \mathbb{R}^n$ be the second and third eigenvectors of L (unit-norm, breaking ties by lexicographic sign). Define the rank-2 spectral pseudo-moment matrix $M_G[i,j] := (v_2[i]v_2[j] + v_3[i]v_3[j])/Z$, where $Z := \max_{\{a,b\}} |v_2[a]v_2[b] + v_3[a]v_3[b]|$, and the TP_2 defect $\delta(G) := |\{(i < j, k < l) : \det M_G[\{i,k\}, \{j,l\}] < -1/n^{3/\binom{n}{2}}\}|$. Conjecture: there exist absolute constants $c_1 = 0.5$ and $c_2 = 0.10$ such that (i) every graph with $\max\text{-CUT}(G)/|E| \geq 0.879$ satisfies $\delta(G) \leq c_1 \cdot n^{-1/2}$, and (ii) every graph on $n \geq 12$ vertices with $\max\text{-CUT}(G)/|E| \leq 0.55$ satisfies $\delta(G) \geq c_2$. A single graph violating either inequality refutes the conjecture.

Rationale (proposer's reasoning):

Pólya frequency / TP_2 matrices are exactly the moment matrices of log-concave one-parameter families; SOS-tight max-CUT instances (where degree-2 SOS already certifies a near-optimal cut) should correspond to log-concave 1D rounding directions and thus preserve TP_2 in the leading spectral block, whereas integrality-gap instances spread mass in a way that flips 2x2 minor signs. Because TP_2 is closed under coordinatewise polynomial gadget tensoring (Karlin 1968, Ch. 3), δ behaves as a lifting-friendly invariant: gadget composition can only grow δ by a constant factor, matching the 'preserved under polynomial reductions' requirement on the target property P. The bridge between Pólya frequency theory and SOS Max-CUT lower bounds appears to have under five papers (TP_k has been used in stable-polynomial proofs of Kadison-Singer but not in SOS degree-lower-bound machinery).

Taxonomy category: LIFTING (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d7213196951c828b

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 4 sizes $n \in \{10, 14, 18, 22\} \times 30$ seeds $\times 5$ families (~ 600 instances), the conjecture is SUPPORTED iff zero falsifiers occur: no instance with $r \geq 0.879$ has $\delta > 0.5/\sqrt{n}$ AND no instance ($n \geq 12$) with $r \leq 0.55$ has $\delta < 0.10$. Additionally Pearson $\text{corr}(\delta, 1-r) \geq 0.3$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.86	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - total positivity TP2 Laplacian eigenvectors Max-Cut SOS - sign regularity minors pseudo-moment Goemans-Williamson semidefinite - Polya frequency spectral embedding Max-Cut degree-2 sum-of-squares

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=3.2s

5.1 Generated Python source

```
import sys
import random
import math
import itertools
from collections import defaultdict

def matrix_multiply(A, B):
    n = len(A)
    m = len(B[0])
    p = len(B)
    result = [[0.0 for _ in range(m)] for _ in range(n)]
    for i in range(n):
        for j in range(m):
            for k in range(p):
                result[i][j] += A[i][k] * B[k][j]
    return result

def matrix_transpose(A):
```

```

    return [list(row) for row in zip(*A)]

def matrix_subtract(A, B):
    return [[A[i][j] - B[i][j] for j in range(len(A[0]))] for i in range(len(A))]

def matrix_add(A, B):
    return [[A[i][j] + B[i][j] for j in range(len(A[0]))] for i in range(len(A))]

def matrix_scale(A, scalar):
    return [[A[i][j] * scalar for j in range(len(A[0]))] for i in range(len(A))]

def matrix_norm(A):
    return math.sqrt(sum(sum(a**2 for a in row) for row in A))

def matrix_identity(n):
    return [[1.0 if i == j else 0.0 for j in range(n)] for i in range(n)]

def matrix_power(A, power):
    result = matrix_identity(len(A))
    for _ in range(power):
        result = matrix_multiply(result, A)
    return result

def matrix_trace(A):
    return sum(A[i][i] for i in range(len(A)))

def matrix_determinant(A):
    n = len(A)
    if n == 1:
        return A[0][0]
    det = 0.0
    for col in range(n):
        minor = [row[:col] + row[col+1:] for row in A[1:]]
        det += ((-1)**col) * A[0][col] * matrix_determinant(minor)
    return det

def matrix_inverse(A):
    n = len(A)
    inverse = [[0.0 for _ in range(n)] for _ in range(n)]
    det = matrix_determinant(A)
    if det == 0:
        raise ValueError("Matrix is not invertible")
    for i in range(n):
        for j in range(n):
            minor = [row[:j] + row[j+1:] for row in (A[:i] + A[i+1:])]
            inverse[j][i] = ((-1)**(i+j)) * matrix_determinant(minor) / det
    return inverse

def matrix_eigen(A, max_iter=1000, tol=1e-10):
    n = len(A)
    eigenvectors = matrix_identity(n)
    for _ in range(max_iter):
        Q, R = qr_decomposition(A)
        A = matrix_multiply(R, Q)
        eigenvectors = matrix_multiply(eigenvectors, Q)
        if matrix_norm(matrix_subtract(A, matrix_scale(matrix_identity(n), A[0][0]))) < tol:

```

```

        break
    eigenvalues = [A[i][i] for i in range(n)]
    return eigenvalues, eigenvectors

def qr_decomposition(A):
    n = len(A)
    Q = [[0.0 for _ in range(n)] for _ in range(n)]
    R = [[0.0 for _ in range(n)] for _ in range(n)]
    for k in range(n):
        R[k][k] = math.sqrt(sum(A[i][k]**2 for i in range(k, n)))
        Q[k][k] = 1.0
        for i in range(k+1, n):
            Q[i][k] = A[i][k] / R[k][k]
            for j in range(k+1, n):
                R[k][j] += Q[i][k] * A[i][j]
    return Q, R

def generate_erdos_renyi(n, p):
    graph = [[0 for _ in range(n)] for _ in range(n)]
    for i in range(n):
        for j in range(i+1, n):
            if random.random() < p:
                graph[i][j] = graph[j][i] = 1
    return graph

def generate_random_regular(n, d):
    if n * d % 2 != 0:
        raise ValueError("n * d must be even")
    graph = [[0 for _ in range(n)] for _ in range(n)]
    edges = []
    for i in range(n):
        for _ in range(d):
            edges.append(i)
    random.shuffle(edges)
    for i in range(0, len(edges), 2):
        u, v = edges[i], edges[i+1]
        if u != v and graph[u][v] == 0:
            graph[u][v] = graph[v][u] = 1
    return graph

def generate_complete_graph(n):
    return [[1 if i != j else 0 for j in range(n)] for i in range(n)]

def generate_random_bipartite(n, p):
    graph = [[0 for _ in range(n)] for _ in range(n)]
    for i in range(n//2):
        for j in range(n//2, n):
            if random.random() < p:
                graph[i][j] = graph[j][i] = 1
    return graph

def generate_planted_bisection(n, noise):
    graph = [[0 for _ in range(n)] for _ in range(n)]
    for i in range(n//2):
        for j in range(n//2, n):
            graph[i][j] = graph[j][i] = 1

```

```

for i in range(n):
    for j in range(i+1, n):
        if random.random() < noise:
            graph[i][j] = 1 - graph[i][j]
            graph[j][i] = graph[i][j]
return graph

def combinatorial_laplacian(graph):
    n = len(graph)
    D = [[0.0 for _ in range(n)] for _ in range(n)]
    A = [[float(x) for x in row] for row in graph]
    for i in range(n):
        D[i][i] = sum(A[i])
    L = matrix_subtract(D, A)
    return L

def max_cut_ratio(graph):
    n = len(graph)
    max_cut = 0
    for mask in range(1 << n):
        cut = 0
        for i in range(n):
            for j in range(i+1, n):
                if ((mask >> i) & 1) != ((mask >> j) & 1):
                    cut += graph[i][j]
        if cut > max_cut:
            max_cut = cut
    edge_count = sum(sum(row) for row in graph) // 2
    if edge_count == 0:
        return 0.0
    return max_cut / edge_count

def spectral_pseudo_moment_matrix(graph):
    L = combinatorial_laplacian(graph)
    eigenvalues, eigenvectors = matrix_eigen(L)
    sorted_indices = sorted(range(len(eigenvalues)), key=lambda i: eigenvalues[i])
    v2 = [eigenvectors[i][sorted_indices[1]] for i in range(len(graph))]
    v3 = [eigenvectors[i][sorted_indices[2]] for i in range(len(graph))]
    Z = max(abs(v2[i]*v2[j] + v3[i]*v3[j]) for i in range(len(graph)) for j in range(len(graph)))
    if Z == 0:
        Z = 1.0
    M = [[(v2[i]*v2[j] + v3[i]*v3[j]) / Z for j in range(len(graph))] for i in range(len(graph))]
    return M

def tp2_defect(graph):
    n = len(graph)
    M = spectral_pseudo_moment_matrix(graph)
    defect = 0
    for i, j in itertools.combinations(range(n), 2):
        for k, l in itertools.combinations(range(n), 2):
            minor = [[M[i][k], M[i][l]], [M[j][k], M[j][l]]]
            det = matrix_determinant(minor)
            if det < -1.0 / (n**3):
                defect += 1
    total = math.comb(n, 2)**2
    return defect / total if total > 0 else 0.0

```

```

def run_trial(seed):
    random.seed(seed)
    n_values = [10, 14, 18, 22]
    families = [
        ('erdos_renyi', lambda n: generate_erdos_renyi(n, 0.5)),
        ('random_regular', lambda n: generate_random_regular(n, 3) if n % 2 == 0 else generate_random_regular(n, 3)),
        ('complete', generate_complete_graph),
        ('random_bipartite', lambda n: generate_random_bipartite(n, 0.5)),
        ('planted_bisection', lambda n: generate_planted_bisection(n, 0.1))
    ]
    metric_values = []
    conjecture_holds = True
    counterexample = ""
    instances_tested = 0

    for n in n_values:
        for family_name, family_generator in families:
            graph = family_generator(n)
            r = max_cut_ratio(graph)
            delta = tp2_defect(graph)
            metric_values.append(delta)
            instances_tested += 1

            if r >= 0.879 and delta > 0.5 / math.sqrt(n):
                conjecture_holds = False
                counterexample = f"Graph with r={r} and delta={delta} violates condition (i)"
                break
            if n >= 12 and r <= 0.55 and delta < 0.10:
                conjecture_holds = False
                counterexample = f"Graph with r={r} and delta={delta} violates condition (ii)"
                break
        if not conjecture_holds:
            break

    if len(metric_values) == 0:
        return {
            "metric_name": "TP_2 defect",
            "metric_value": 0.0,
            "instances_tested": 0,
            "conjecture_holds": False,
            "counterexample": "No instances tested"
        }

    mean_delta = sum(metric_values) / len(metric_values)
    return {
        "metric_name": "TP_2 defect",
        "metric_value": mean_delta,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97]
    trials = []

```

```

for seed in seeds:
    result = run_trial(seed)
    trials.append(result)
    print(f"TRIAL: {result}")

if not trials:
    print("RESULT: INCONCLUSIVE reason=no_trials")
    sys.exit(0)

metric_values = [trial["metric_value"] for trial in trials if trial["metric_value"] is not None]
if not metric_values:
    print("RESULT: INCONCLUSIVE reason=no_metric_values")
    sys.exit(0)

mean_metric = sum(metric_values) / len(metric_values)
std_metric = math.sqrt(sum((x - mean_metric)**2 for x in metric_values) / len(metric_values))
support_fraction = sum(1 for trial in trials if trial["conjecture_holds"]) / len(trials)

if all(trial["conjecture_holds"] for trial in trials):
    print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fraction}")
elif any(not trial["conjecture_holds"] for trial in trials):
    first_failing_seed = next(trial["seed"] for trial in trials if not trial["conjecture_holds"])
    counterexample = next(trial["counterexample"] for trial in trials if not trial["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

TRIAL: {'metric_name': 'TP_2 defect', 'metric_value': 0.0004911287328869746, 'instances_tested': 8,
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_41095b21.py", line 253, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_41095b21.py", line 216, in run_trial
    delta = tp2_defect(graph)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_41095b21.py", line 186, in tp2_defect
    M = spectral_pseudo_moment_matrix(graph)
        ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_41095b21.py", line 174, in spectral_pseudo_moment_matrix
    eigenvalues, eigenvectors = matrix_eigen(L)
                                   ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_41095b21.py", line 80, in matrix_eigen
    Q, R = qr_decomposition(A)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_41095b21.py", line 96, in qr_decomposition
    Q[i][k] = A[i][k] / R[k][k]
              ~~~~~
ZeroDivisionError: float division by zero

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a ZeroDivisionError in the custom QR decomposition after only 8 instances, well short of the pre-registered ~600 instance protocol. Although a putative low-cut counterexample was reported, the run terminated abnormally and the support condition cannot be evaluated. | next: Replace the hand-rolled QR with `numpy.linalg.eigh` (or guard against zero pivots) and rerun the full $4 \times 30 \times 5$ protocol before judging the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	143938
2	preregistration	claude_max	opus	0	0	7579
3	novelty	claude_max	opus	0	0	3133
4	novelty	claude_max	opus	0	0	11266
5	test_gen	mistral	codestral-latest	0	0	324569
6	test_gen	mistral	codestral-latest	0	0	320604
7	test_gen	mistral	codestral-latest	0	0	314620
8	test_gen	mistral	codestral-latest	0	0	319874
9	judge	claude_max	opus	0	0	4697

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 1450280 ms total latency. Provider mix: {'claude_max': 5, 'mistral': 4}

(full prompt+response transcripts available in `research/audit/16809e692923.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/16809e692923.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/16809e692923.tar.gz` (if generated)